

HTML SVG

A Practical, From-Zero Learning Guide

Part 7 — Reuse, Symbols, Markers & Advanced Structure

Large SVG graphics become much easier to manage when you stop drawing the same thing repeatedly. SVG provides definitions, reusable instances, symbols and markers so you can build efficient icon systems, diagrams and data visualizations.

Goal: Learn how to define graphics once and reuse them many times, create arrowheads and markers, and organize SVG like a reusable component library.

1. Why Reuse Matters

Imagine a map containing 500 identical location icons. Copying the entire icon 500 times makes the SVG larger and harder to maintain. SVG lets you define an icon once and reference it repeatedly.

```
<defs>
  <!-- define reusable artwork here -->
</defs>
```

<defs> is commonly used for resources that are defined but not directly rendered until referenced.

2. The Element

<use> creates an instance of previously defined SVG content.

```
<svg viewBox="0 0 300 150">

  <defs>
    <circle id="dot"
      cx="20" cy="20" r="12"
      fill="tomato"/>
  </defs>

  <use href="#dot"/>
  <use href="#dot" x="60"/>
  <use href="#dot" x="120"/>
  <use href="#dot" x="180"/>

</svg>
```

The definition appears once, while **<use>** creates multiple instances.

3. Moving a Reused Element

```
<defs>
  <rect id="box"
    width="50" height="50"
    rx="8"
    fill="royalblue"/>
</defs>

<use href="#box" x="20" y="20"/>
<use href="#box" x="90" y="60"/>
<use href="#box" x="160" y="20"/>
```

The x and y attributes on **<use>** move the instance.

4. Scaling a Reused Element

```
<defs>
  <circle id="ball"
    cx="0" cy="0" r="20"
    fill="orange"/>
</defs>

<use href="#ball"
  x="60" y="60"/>

<use href="#ball"
  x="130" y="60"
  transform="scale(2)"/>
```

Transformations can be applied to the instance. Be careful with coordinate systems when scaling around an unexpected origin.

5. Styling Reused Content

```

<defs>
  <circle id="icon"
    cx="10" cy="10" r="10"/>
</defs>

<use href="#icon"
  x="30" y="30"
  fill="red"/>

<use href="#icon"
  x="70" y="30"
  fill="blue"/>

```

Presentation properties can often be inherited or overridden on the reused instance, making reusable artwork flexible.

6. The Element

<symbol> is designed specifically for reusable SVG graphics. A symbol can have its own `viewBox` and is normally rendered through **<use>**.

```

<svg viewBox="0 0 300 150">

  <symbol id="heart"
    viewBox="0 0 100 100">
    <path d="M50 85
      L15 50
      C-5 30 10 5 35 15
      C45 19 50 30 50 30
      C50 30 55 19 65 15
      C90 5 105 30 85 50
      Z"
      fill="currentColor"/>
    </symbol>

    <use href="#heart"
      x="30" y="30"
      width="70" height="70"/>

</svg>

```

Symbols are particularly useful for icon systems.

7. Why Symbols Are Useful

- Each symbol can define a reusable `viewBox`.
- You can render the same symbol at different sizes.
- You can organize many icons inside one SVG sprite.
- You avoid copying complex path data.
- One definition can be updated and reused everywhere.

8. symbol viewBox and Instance Size

```

<symbol id="star"
  viewBox="0 0 100 100">
  <path d="..." />
</symbol>

<use href="#star"
  x="20" y="20"
  width="50" height="50"/>

<use href="#star"
  x="100" y="20"
  width="100" height="100"/>

```

The symbol's viewBox establishes its internal coordinate system while the use element controls the displayed size.

9. SVG Sprites

An SVG sprite is a collection of reusable symbols, often used for website icons.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="search" viewBox="0 0 24 24">
    <circle cx="11" cy="11" r="7"
      fill="none"
      stroke="currentColor"
      stroke-width="2"/>
    <path d="M16 16 L22 22"
      fill="none"
      stroke="currentColor"
      stroke-width="2"/>
  </symbol>

  <symbol id="close" viewBox="0 0 24 24">
    <path d="M5 5 L19 19 M19 5 L5 19"
      fill="none"
      stroke="currentColor"
      stroke-width="2"/>
  </symbol>
</svg>
```

The sprite acts like an icon library.

10. Using a Sprite Symbol

```
<svg viewBox="0 0 24 24"
  width="32" height="32">
  <use href="#search"/>
</svg>
```

When the symbols are available in the document, **<use>** references the desired icon.

11. External SVG Sprites

A symbol can also be referenced from another SVG resource in supporting environments.

```
<svg viewBox="0 0 24 24"
  width="32" height="32">
  <use href="icons.svg#search"/>
</svg>
```

External SVG usage can have browser, security and deployment considerations, so test it in the environment where your application will run.

12. vs

Feature	<defs>	<symbol>
Main purpose	Store reusable resources	Define reusable SVG graphics
Rendered directly	Usually no	No, normally via use
Own viewBox	No	Yes
Ideal for	Gradients, clips, filters, shapes	Icons and reusable graphics
Common with use	Yes	Yes

13. Markers — The Basic Idea

Markers place a predefined graphic at points along paths, lines and polygons. They are commonly used for arrowheads.

```
<defs>
  <marker id="arrow"
    markerWidth="10"
    markerHeight="10"
    refX="8"
    refY="5"
    orient="auto">
    <path d="M0 0 L10 5 L0 10 Z"
      fill="black"/>
  </marker>
</defs>

<path d="M30 100 H260"
  stroke="black"
  stroke-width="4"
  fill="none"
  marker-end="url(#arrow)"/>
```

The marker is defined once and attached using **marker-end**.

14. marker-start and marker-mid

```
<path d="M40 100
  L140 30
  L240 100"
  fill="none"
  stroke="black"
  stroke-width="4"
  marker-start="url(#arrow)"
  marker-mid="url(#arrow)"
  marker-end="url(#arrow)"/>
```

Markers can appear at the start, middle vertices and end of supported path geometry.

15. markerWidth and markerHeight

These attributes define the marker's viewport size.

```
<marker id="arrow"
  markerWidth="12"
  markerHeight="12">
  ...
</marker>
```

16. refX and refY

The reference point determines which point of the marker is attached to the path.

```
<marker id="arrow"
  markerWidth="12"
  markerHeight="12"
  refX="10"
  refY="6">
  <path d="M0 0 L12 6 L0 12 Z"/>
</marker>
```

Changing refX/refY changes how the arrowhead sits relative to the path endpoint.

17. orient

The **orient** attribute controls marker rotation.

```
<marker id="arrow"
  markerWidth="10"
  markerHeight="10"
  refX="8"
```

```

        refY="5"
        orient="auto">
    <path d="M0 0 L10 5 L0 10 Z"/>
</marker>

```

auto automatically follows the direction of the path. There are also fixed-angle values such as **0** and **45** degrees.

18. markerUnits

```

<marker id="arrow"
    markerWidth="10"
    markerHeight="10"
    markerUnits="strokeWidth">
    ...
</marker>

```

With **strokeWidth**, marker dimensions scale with the stroke width. With **userSpaceOnUse**, the marker uses the SVG user coordinate system.

19. Arrow on a Curved Path

```

<defs>
  <marker id="arrow"
    markerWidth="12"
    markerHeight="12"
    refX="10"
    refY="6"
    orient="auto">
    <path d="M0 0 L12 6 L0 12 Z"/>
  </marker>
</defs>

<path d="M30 150
  Q150 10 270 150"
  fill="none"
  stroke="royalblue"
  stroke-width="5"
  marker-end="url(#arrow)"/>

```

The arrowhead automatically rotates to follow the path direction.

20. Markers for Diagrams

Markers are ideal for flowcharts, timelines, network graphs, maps, directional diagrams and relationship lines.

21. Reusable Component Pattern

A useful SVG architecture is:

- Define reusable resources inside **defs**.
- Define icons as **symbols**.
- Use **use** to create instances.
- Define arrowheads and other endpoint decorations as **markers**.
- Keep IDs descriptive and unique.

```

<svg viewBox="0 0 500 300">

  <defs>
    <!-- gradients -->
    <!-- filters -->
    <!-- clip paths -->
    <!-- reusable shapes -->
  </defs>

```

```

<symbol id="icon">
  <!-- icon geometry -->
</symbol>

<marker id="arrow">
  <!-- arrow geometry -->
</marker>

<!-- instances -->
<use href="#icon" .../>
<use href="#icon" .../>

</svg>

```

22. Reusing a Complete Group

```

<defs>
  <g id="house">
    <rect x="20" y="50"
          width="80" height="60"
          fill="#f59e0b"/>
    <path d="M10 50 L60 10 L110 50 Z"
          fill="#ef4444"/>
  </g>
</defs>

<use href="#house" x="0" y="0"/>
<use href="#house" x="130" y="20"/>
<use href="#house" x="260" y="0"/>

```

You can define a complete group and reuse it as a unit.

23. Instance-Specific Transformations

```

<use href="#house"
    x="20" y="20"
    transform="scale(0.8)"/>

<use href="#house"
    x="200" y="20"
    transform="scale(1.3)"/>

```

Each instance can have its own transform, allowing one definition to produce many variations.

24.currentColor + Symbols

```

<symbol id="heart"
  viewBox="0 0 100 100">
  <path d="..."
    fill="currentColor"/>
</symbol>

<svg width="40" height="40"
  color="tomato">
  <use href="#heart"/>
</svg>

<svg width="40" height="40"
  color="royalblue">
  <use href="#heart"/>
</svg>

```

This is a strong pattern for themeable icon libraries.

25. Icon System Example

```

<svg xmlns="http://www.w3.org/2000/svg">

  <defs>
    <symbol id="home" viewBox="0 0 24 24">

```

```

    <path d="M3 11 L12 3 L21 11 V21 H3 Z"
        fill="none"
        stroke="currentColor"
        stroke-width="2"/>
</symbol>

<symbol id="user" viewBox="0 0 24 24">
  <circle cx="12" cy="8" r="4"
        fill="none"
        stroke="currentColor"
        stroke-width="2"/>
  <path d="M4 21
        C4 16 20 16 20 21"
        fill="none"
        stroke="currentColor"
        stroke-width="2"/>
</symbol>
</defs>

</svg>

```

A project can keep many symbols together as its own icon collection.

26. Accessibility with Reused Icons

Decorative icons should generally not be announced unnecessarily. Meaningful standalone graphics can provide accessible names.

```

<svg role="img"
    aria-labelledby="iconTitle">
  <title id="iconTitle">Home</title>
  <use href="#home"/>
</svg>

```

For UI icons next to visible text, the visible text may already provide the meaning, depending on how the component is structured.

27. IDs Matter

SVG references such as `url(#gradient)` and `href="#icon"` depend on IDs.

- Keep IDs unique within the relevant document.
- Use descriptive names.
- Avoid accidental duplicate IDs when generating SVG dynamically.
- When reusing SVG fragments in JavaScript applications, be careful about ID collisions.

28. SVG in React

The same reuse ideas apply when SVG is used inside React. You can create components around SVG symbols or simply create reusable React icon components.

```

function SearchIcon({size = 24}) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
    >
      <circle cx="11" cy="11" r="7" />
      <path d="M16 16 L22 22" />
    </svg>
  );
}

```


This is especially useful in frontend projects because SVG can follow React props such as `size`, `className` and `color`.

29. Mini Project — Icon Grid

```
<svg viewBox="0 0 500 180">

  <symbol id="star"
    viewBox="0 0 100 100">
    <path d="M50 5
      L61 38
      L95 38
      L68 58
      L78 92
      L50 72
      L22 92
      L32 58
      L5 38
      L39 38 Z"
      fill="currentColor"/>
    </symbol>

  <use href="#star"
    x="30" y="50"
    width="50" height="50"
    color="#f59e0b"/>

  <use href="#star"
    x="110" y="50"
    width="70" height="70"
    color="#ef4444"/>

  <use href="#star"
    x="210" y="40"
    width="90" height="90"
    color="#7c3aed"/>

</svg>
```

30. Mini Project — Flow Diagram

```
<svg viewBox="0 0 600 250">

  <defs>
    <marker id="arrow"
      markerWidth="10"
      markerHeight="10"
      refX="8"
      refY="5"
      orient="auto">
      <path d="M0 0 L10 5 L0 10 Z"
        fill="black"/>
    </marker>
  </defs>

  <rect x="30" y="80"
    width="120" height="60"
    rx="12" fill="#dbeafe"/>

  <rect x="240" y="80"
    width="120" height="60"
    rx="12" fill="#dcfce7"/>

  <rect x="450" y="80"
    width="120" height="60"
    rx="12" fill="#fef3c7"/>

  <path d="M150 110 H240"
    stroke="black"
    stroke-width="4"
    marker-end="url(#arrow)"/>
```

```

<path d="M360 110 H450"
      stroke="black"
      stroke-width="4"
      marker-end="url(#arrow)"/>

<text x="90" y="115"
      text-anchor="middle">Start</text>

<text x="300" y="115"
      text-anchor="middle">Process</text>

<text x="510" y="115"
      text-anchor="middle">Finish</text>

</svg>

```

31. Mini Project — Reusable Houses

```

<svg viewBox="0 0 600 220">

  <defs>
    <g id="house">
      <rect x="20" y="70"
            width="100" height="70"
            fill="#f59e0b"/>
      <path d="M10 70 L70 20 L130 70 Z"
            fill="#ef4444"/>
      <rect x="55" y="100"
            width="30" height="40"
            fill="#78350f"/>
    </g>
  </defs>

  <use href="#house" x="0" y="20"/>
  <use href="#house" x="180" y="0"
        transform="scale(1.1)"/>
  <use href="#house" x="380" y="30"
        transform="scale(0.8)"/>

</svg>

```

32. Common Mistakes

- Duplicating complex SVG instead of reusing it.
- Using duplicate IDs.
- Confusing **symbol** with a normal visible shape.
- Forgetting to set a useful viewBox on symbols.
- Placing a marker incorrectly because refX/refY are misunderstood.
- Using markerUnits without understanding how marker size scales.
- Assuming an external SVG sprite works identically in every environment.
- Overcomplicating a small icon when a simple path is enough.

33. Exercises

- Create a reusable star with **use** and display it five times.
- Create a symbol-based home icon.
- Build a three-icon SVG sprite.
- Create an arrow marker and use it on a curved path.

- Create a flowchart with three boxes and two arrows.
- Create a reusable house group and build a small city.
- Make a themeable icon using `currentColor`.
- Create a reusable badge symbol and render it at three sizes.
- Build a small SVG icon library for your own website.

34. Quick Reference

Element / Attribute	Purpose
<code><defs></code>	Stores reusable definitions/resources
<code><use></code>	Creates an instance of defined SVG content
<code><symbol></code>	Defines reusable SVG graphics with a <code>viewBox</code>
<code>href="#id"</code>	References SVG content by ID
<code><marker></code>	Defines endpoint/midpoint decorations
<code>marker-start</code>	Marker at path start
<code>marker-mid</code>	Markers at path vertices
<code>marker-end</code>	Marker at path end
<code>markerWidth / markerHeight</code>	Marker viewport size
<code>refX / refY</code>	Marker attachment reference point
<code>orient</code>	Marker rotation
<code>markerUnits</code>	Controls marker coordinate scaling
<code>currentColor</code>	Makes reusable artwork themeable
<code>viewBox</code>	Defines an element's internal coordinate system

35. What's Next?

Part 8 will cover **SVG Paths in Depth**: the complete path command system, relative vs absolute commands, curves, arcs, Bézier mathematics, path construction strategies, path length and advanced drawing projects.